

UNIVERSIDAD SAN FRANCISCO XAVIER DE CHUQUISACA  
FACULTAD DE INGENIERÍA · INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

## COM-350 ARQUITECTURA DE SOFTWARE

# Práctica Guiada N.º 1

## Principios SOLID aplicados a la refactorización

*Caso: Sistema de Biblioteca USFX · Java 17+ · Apache NetBeans · Maven*

| CAMPO              | DETALLE                                                                       |
|--------------------|-------------------------------------------------------------------------------|
| Asignatura         | COM-350 — Arquitectura de Software                                            |
| Tema               | Tema 3 — Diseño de Software (SOLID, acoplamiento y cohesión, refactorización) |
| Tipo de actividad  | Práctica guiada en aula + dos ejercicios evaluados                            |
| Modalidad          | Individual la práctica guiada; en parejas los ejercicios                      |
| Duración estimada  | Práctica guiada: 90 minutos · Ejercicios: 2 sesiones de laboratorio           |
| Lenguaje y entorno | Java 17 o superior · Apache NetBeans 21+ · Apache Maven · Git                 |
| Bibliotecas        | JUnit 5, Mockito, AssertJ y ArchUnit (declaradas en el pom.xml)               |
| Proyecto base      | biblioteca-solid.zip (se entrega junto con esta guía)                         |
| Puntaje total      | 100 puntos (guiada 20 · ejercicio 1 · 40 · ejercicio 2 · 40)                  |

### CRITERIO CENTRAL DE LA PRÁCTICA

Refactorizar es cambiar la estructura interna del código sin alterar su comportamiento observable. Por eso el proyecto llega con una red de pruebas de caracterización ya escrita: si una de ellas se pone en rojo, no refactorizaste — cambiaste el programa.

## 1 OBJETIVOS Y RESULTADOS DE APRENDIZAJE

Al terminar esta práctica el estudiante será capaz de diagnosticar un diseño deficiente, nombrar el principio que se está violando y ejecutar la refactorización concreta que lo corrige, dejando evidencia verificable de cada paso.

### 1.1 Objetivo general

Refactorizar un módulo Java existente aplicando los cinco principios SOLID, sin alterar su comportamiento observable y respaldando cada cambio con pruebas automatizadas y control de versiones.

### 1.2 Objetivos específicos

- Identificar en código real los síntomas de rigidez, fragilidad, inmovilidad y complejidad innecesaria descritos en el Tema 3.
- Asociar cada síntoma con el principio SOLID que lo explica: SRP, OCP, LSP, ISP o DIP.
- Aplicar las refactorizaciones del catálogo de Fowler: Extract Class, Replace Conditional with Polymorphism, Extract Interface, Replace Constructor with Dependency Injection.
- Escribir pruebas unitarias con JUnit 5 y dobles de prueba con Mockito que solo son posibles cuando el diseño respeta DIP.
- Convertir reglas de diseño en pruebas ejecutables con ArchUnit y usarlas como criterio objetivo de "terminado".
- Documentar la refactorización con un historial de commits legible, un commit por paso.

### 1.3 Resultados verificables (lo que se evalúa)

| CÓD. | RESULTADO                                                       | CÓMO SE VERIFICA                                              |
|------|-----------------------------------------------------------------|---------------------------------------------------------------|
| R1   | Las pruebas de caracterización siguen en verde tras cada paso   | Captura de la pestaña Test Results de NetBeans en cada commit |
| R2   | Cada principio SOLID se aplicó con una refactorización nombrada | Tabla olor → principio → refactorización del informe          |
| R3   | El comportamiento observable no cambió                          | Diff de la salida de Main antes y después                     |
| R4   | El historial de commits refleja los pasos                       | Salida de git log --oneline                                   |
| R5   | Las reglas de ArchUnit pasan de rojo a verde                    | Par de capturas antes/después del Paso 6                      |
| R6   | Se puede agregar una variante nueva sin editar código existente | Prueba de extensión que compila y pasa                        |

## 2 PREPARACIÓN DEL ENTORNO

### 2.1 Requisitos

| HERRAMIENTA            | VERSIÓN MÍN.      | COMPROBACIÓN  |
|------------------------|-------------------|---------------|
| JDK (Temurin u Oracle) | 17 (ideal 21 LTS) | java -version |

| HERRAMIENTA     | VERSIÓN MÍN.            | COMPROBACIÓN  |
|-----------------|-------------------------|---------------|
| Apache NetBeans | 21                      | Help → About  |
| Apache Maven    | 3.8 (el incluido sirve) | mvn -v        |
| Git             | cualquiera              | git --version |

## 2.2 Abrir el proyecto en NetBeans

1. Descomprime biblioteca-solid.zip en una carpeta sin espacios ni tildes en la ruta, por ejemplo D:\practicas\biblioteca-solid.
2. En NetBeans: **File → Open Project...** y selecciona la carpeta. NetBeans reconoce el proyecto por su pom.xml.
3. Clic derecho sobre el proyecto → **Build with Dependencies**. La primera vez Maven descarga JUnit 5, Mockito, AssertJ y ArchUnit; necesitas conexión a internet una sola vez.
4. Clic derecho → **Test** (Alt+F6). Deben aparecer todas las pruebas en verde y la clase ReglasDisenoTest marcada como omitida (skipped).

### Terminal VERIFICACIÓN

```
# Verificación desde la consola, dentro de la carpeta del proyecto
mvn -v
mvn clean test

# Salida esperada al final:
# Tests run: 37, Failures: 0, Errors: 0, Skipped: <la clase ReglasDisenoTest>
# BUILD SUCCESS
```

## 2.3 Inicializar el control de versiones

Todo el historial de la refactorización es parte de la evidencia. Antes de tocar una sola línea, crea el repositorio y registra la línea base:

### Git OBLIGATORIO

```
cd biblioteca-solid
git init
git add .
git commit -m "paso 0: línea base, código legado con pruebas de caracterización en verde"
```

### SI TRABAJAS SIN INTERNET EN EL AULA

Ejecuta una vez en casa "mvn clean test" para que Maven guarde las dependencias en tu repositorio local (~/.m2). A partir de ahí el proyecto compila sin conexión.

### 3 EL CASO: MÓDULO DE PRÉSTAMOS DE LA BIBLIOTECA USFX

El Sistema de Biblioteca de la universidad tiene un módulo de préstamos escrito hace tres gestiones por un grupo de pasantes. Funciona. Está en producción. Y cada solicitud de cambio de la Dirección de Bibliotecas termina en el mismo archivo: `GestorBiblioteca.java`.

Durante el último semestre llegaron estas solicitudes:

| SOLICITA                 | PEDIDO                                                                    | QUÉ HAY QUE TOCAR HOY |
|--------------------------|---------------------------------------------------------------------------|-----------------------|
| Dirección de Bibliotecas | Crear la categoría "EGRESADO" con 5 días de plazo y multa de Bs 3 por día | GestorBiblioteca      |
| Dirección de TI          | Migrar de MySQL a PostgreSQL                                              | GestorBiblioteca      |
| Comunicación             | Cambiar el correo por notificación WhatsApp                               | GestorBiblioteca      |
| Kardex                   | Emitir el reporte mensual en Excel, no en CSV                             | GestorBiblioteca      |
| Biblioteca Central       | Registrar obras de referencia que solo se consultan en sala               | GestorBiblioteca      |
| Hemeroteca               | Prestar material digital con licencia por descarga                        | GestorBiblioteca      |

#### EL DIAGNÓSTICO ES INMEDIATO

Seis actores distintos piden cambios sobre el mismo archivo. Eso es exactamente lo que el SRP define como violación: un módulo responsable ante más de un actor. Los otros cuatro principios explican por qué cada uno de esos cambios es caro.

#### 3.1 Diagnóstico previo (hazlo antes de tocar el código)

Abre `GestorBiblioteca.java` y completa esta tabla en tu informe. Es la evidencia E-1 y vale puntos por sí misma.

| SÍNTOMA OBSERVADO                                         | EVIDENCIA (ARCHIVO : LÍNEA)                | PRINCIPIO VIOLADO | PASO   |
|-----------------------------------------------------------|--------------------------------------------|-------------------|--------|
| Cadena de if/else sobre el tipo de usuario                | <code>GestorBiblioteca.java</code> : 47-60 | OCP               | Paso 2 |
| La clase construye su propia conexión MySQL               | <code>GestorBiblioteca.java</code> : 33-34 | DIP               | Paso 5 |
| Cálculo, persistencia, correo y formato en un método      | <code>registrarPrestamo()</code>           | SRP               | Paso 1 |
| <code>UnsupportedOperationException</code> en un override | <code>LibroReferencia.java</code>          | LSP               | Paso 3 |
| Implementadores que rechazan la mitad de la interfaz      | <code>OperacionesBiblioteca.java</code>    | ISP               | Paso 4 |
| (completa dos filas más por tu cuenta)                    |                                            |                   |        |

## 4 PARTE I — PRÁCTICA GUIADA

Los siete pasos se ejecutan en orden. Cada paso termina con un commit y con una evidencia que se adjunta al informe. No avances al siguiente paso si las pruebas no están en verde.

| PASO | QUÉ SE HACE                            | PRINCIPIO   | EVIDENCIA                             |
|------|----------------------------------------|-------------|---------------------------------------|
| 0    | Línea base y red de seguridad          | —           | E-0 salida de Main + suite en verde   |
| 1    | Separar los cuatro actores             | SRP         | E-1 tabla de actores + nuevas clases  |
| 2    | Reemplazar el if/else por polimorfismo | OCP         | E-2 categoría nueva sin editar código |
| 3    | Arreglar la jerarquía de materiales    | LSP         | E-3 cliente sin instanceof            |
| 4    | Partir la interfaz gorda               | ISP         | E-4 métrica de métodos no soportados  |
| 5    | Invertir las dependencias              | DIP         | E-5 prueba con Mockito sin BD ni SMTP |
| 6    | Verificación automática                | ADP / todos | E-6 ArchUnit de rojo a verde          |

### Paso 0 · Línea base y red de seguridad

Antes de refactorizar hay que saber qué hace el programa hoy. Las pruebas de caracterización de Michael Feathers no juzgan si el comportamiento es correcto: lo congelan.

#### Terminal PASO 0

```
# 1. Ejecuta la demostración y guarda la salida como línea base
mvn -q compile exec:java -Dexec.mainClass=bo.edu.usfx.biblioteca.Main > base.txt

# En NetBeans: clic derecho sobre el proyecto → Run (F6)

# 2. Ejecuta la suite completa
mvn clean test
```

Abre PruebasCaracterizacionTest.java y lee las seis pruebas. Fíjate en lo que congelan: el plazo de 7 días del estudiante, el formato exacto del comprobante, el tope de multa de Bs 200 y la cabecera del CSV. Ese es el contrato que no puedes romper.

#### PruebasCaracterizacionTest.java RED DE SEGURIDAD

```
@ParameterizedTest(name = "{0} con {1} días de retraso paga Bs {2}")
@CsvSource({ "ESTUDIANTE, 5, 10.0", "DOCENTE, 5, 5.0",
             "ADMINISTRATIVO, 5, 7.5", "EXTERNO, 5, 25.0",
             "EXTERNO, 100, 200.0" }) // tope de 200 Bs
void tarifaDeMulta(String tipo, int diasRetraso, double esperado) {
    GestorBiblioteca gestor = new GestorBiblioteca();
    Usuario usuario = new Usuario("999", "Prueba", "p@usfx.bo", tipo);
    Prestamo prestamo = new Prestamo(usuario, libro(), HOY, HOY.plusDays(7));

    double multa = gestor.calcularMulta(prestamo, HOY.plusDays(7 + diasRetraso));

    assertThat(multa).isEqualTo(esperado);
}
```

## EVIDENCIA E-0

### Línea base

Captura de pantalla de la ventana Output de NetBeans con la salida de Main.

Captura de la pestaña Test Results: todas las pruebas en verde, 0 fallas, con ReglasDisenoTest marcada como omitida.

Salida de git log --oneline con el commit "paso 0: linea base...".

## Paso 1 · SRP — un módulo, un actor

| CAMPO           | CONTENIDO                                                                                                  |
|-----------------|------------------------------------------------------------------------------------------------------------|
| Síntoma         | El método registrarPrestamo() calcula reglas de negocio, arma SQL, envía correo y formatea un comprobante. |
| Principio       | Single Responsibility Principle — «un módulo debe ser responsable ante un único actor».                    |
| Refactorización | Extract Class (Fowler) — separar en cuatro colaboradores, uno por actor.                                   |

### Identifica los actores

Antes de escribir código, nombra a quién le responde cada bloque. Este es el ejercicio conceptual del SRP y suele ser el paso donde más se equivoca la gente: la pregunta no es "¿qué hace?", sino "¿quién pide el cambio?".

| ACTOR                    | BLOQUE DEL CÓDIGO         | CLASE DESTINO                            |
|--------------------------|---------------------------|------------------------------------------|
| Dirección de Bibliotecas | Plazos, límites y tarifas | dominio/PoliticaPrestamo (Paso 2)        |
| Dirección de TI          | INSERT / UPDATE, conexión | infraestructura/RepositorioPrestamosJdbc |
| Comunicación             | Envío de correos          | infraestructura/NotificadorSmtip         |
| Kardex                   | Comprobante y reporte CSV | presentacion/ComprobantePrestamo         |
| (orquestador)            | La secuencia de pasos     | aplicacion/ServicioPrestamos             |

### Estructura de paquetes objetivo

#### Árbol de paquetes al terminar la práctica

```
bo.edu.usfx.biblioteca
├── dominio                                reglas del negocio, sin tecnología
│   ├── Usuario.java Libro.java Prestamo.java
│   ├── PoliticaPrestamo.java             (interfaz, Paso 2)
│   ├── RepositorioPrestamos.java         (puerto, Paso 5)
│   └── Notificador.java                   (puerto, Paso 5)
├── aplicacion
│   └── ServicioPrestamos.java             orquesta el caso de uso
├── infraestructura                       adaptadores: aquí SÍ hay tecnología
│   ├── RepositorioPrestamosJdbc.java
│   └── NotificadorSmtip.java
└── presentacion
    ├── ComprobantePrestamo.java
    └── ReportePrestamosCsv.java
```

### Antes

#### legado/GestorBiblioteca.java VIOLA SRP

```
public String registrarPrestamo(Usuario usuario, Libro libro, LocalDate hoy) {
    int diasPermitidos; int maximoLibros;
    if ("ESTUDIANTE".equals(usuario.getTipo())) { ... } // ← negocio
    ...
    conexion.ejecutar("INSERT INTO prestamo (...) VALUES ('" // ← persistencia
        + usuario.getCodigo() + "', ...");

    correo.enviar(usuario.getCorreo(), "Prestamo registrado", // ← notificación
        "Estimado/a " + usuario.getNombre() + " ...");

    return "=== BIBLIOTECA USFX ===\n" // ← presentación
        + "Usuario : " + usuario.getNombre() + " ...";
}
```

}

## Después

### aplicacion/ServicioPrestamos.java CUMPLE SRP

```
package bo.edu.usfx.biblioteca.aplicacion;

public class ServicioPrestamos {

    private final RepositorioPrestamos repositorio;
    private final Notificador notificador;
    private final PoliticaPrestamo politica;

    public ServicioPrestamos(RepositorioPrestamos repositorio,
                             Notificador notificador,
                             PoliticaPrestamo politica) {
        this.repositorio = repositorio;
        this.notificador = notificador;
        this.politica = politica;
    }

    public Prestamo registrar(Usuario usuario, Libro libro, LocalDate hoy) {
        politica.validar(usuario, repositorio.activosDe(usuario), libro);
        Prestamo prestamo = new Prestamo(usuario, libro,
                                          hoy, politica.fechaLimite(hoy));
        repositorio.guardar(prestamo);
        notificador.notificar(usuario.getCorreo(),
                              "Préstamo registrado",
                              "Devuelva hasta el " + prestamo.getFechaLimite());
        return prestamo;
    }
}

// El formato del comprobante ya NO vive aquí:
// presentacion/ComprobantePrestamo.imprimir(prestamo)
```

### CUIDADO CON EL EFECTO DOMINÓ

Al mover el formato del comprobante a ComprobantePrestamo, la prueba formatoDelComprobante() deja de compilar. Eso NO significa que puedas borrarla: adáptala para que llame a la clase nueva y siga verificando exactamente la misma cadena. El comportamiento observable no cambió; cambió quién lo produce.

### Terminal COMMIT DEL PASO 1

```
mvn test
git add .
git commit -m "paso 1 (SRP): Extract Class - separar dominio, persistencia, notificacion y presentacion"
```

### EVIDENCIA E-1

#### Separación por actores

La tabla actor → bloque → clase destino completa, con números de línea reales.

Captura del explorador de proyectos de NetBeans mostrando el árbol de paquetes nuevo.

Captura de Test Results en verde después del commit.



## Paso 2 · OCP — extender sin editar

| CAMPO           | CONTENIDO                                                                                                                                                                                                      |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Síntoma         | Dos cadenas de if/else sobre usuario.getTipo(): una decide plazos y límites, otra decide la tarifa de la multa. Cada convenio nuevo obliga a volver al mismo archivo y a re-probar todos los casos anteriores. |
| Principio       | Open-Closed Principle — abierto a la extensión, cerrado a la modificación.                                                                                                                                     |
| Refactorización | Replace Conditional with Polymorphism (Fowler) + Strategy.                                                                                                                                                     |

### Antes

#### legado/GestorBiblioteca.java VIOLA OCP

```

if ("ESTUDIANTE".equals(usuario.getTipo())) { diasPermitidos = 7; maximoLibros = 3; }
else if ("DOCENTE".equals(usuario.getTipo())) { diasPermitidos = 15; maximoLibros = 5; }
else if ("ADMINISTRATIVO".equals(...)) { diasPermitidos = 10; maximoLibros = 2; }
else if ("EXTERNO".equals(...)) { diasPermitidos = 3; maximoLibros = 1; }
else throw new IllegalArgumentException(...);

// ...y cien líneas más abajo, la MISMA cadena otra vez para la multa:
if ("ESTUDIANTE".equals(tipo)) multa = diasRetraso * 2.0;
else if ("DOCENTE".equals(tipo)) multa = diasRetraso * 1.0;
else if ("ADMINISTRATIVO".equals(tipo)) multa = diasRetraso * 1.5;
else if ("EXTERNO".equals(tipo)) multa = diasRetraso * 5.0;

```

### Después

#### dominio/PoliticaPrestamo.java CUMPLE OCP

```

package bo.edu.usfx.biblioteca.dominio;

public interface PoliticaPrestamo {

    boolean aplicaA(Usuario usuario);

    int diasPermitidos();
    int maximoEjemplares();
    BigDecimal tarifaDiaria();

    default BigDecimal multa(long diasRetraso) {
        if (diasRetraso <= 0) return BigDecimal.ZERO;
        BigDecimal calculada = tarifaDiaria()
            .multiply(BigDecimal.valueOf(diasRetraso));
        return calculada.min(TOPE); // el tope sigue siendo Bs 200
    }

    BigDecimal TOPE = new BigDecimal("200");
}

public class PoliticaEstudiante implements PoliticaPrestamo {
    public boolean aplicaA(Usuario u) { return "ESTUDIANTE".equals(u.getTipo()); }
    public int diasPermitidos() { return 7; }
    public int maximoEjemplares() { return 3; }
    public BigDecimal tarifaDiaria() { return new BigDecimal("2.0"); }
}

// El selector reemplaza a las dos cadenas de if/else
public class CatalogoPoliticass {
    private final List<PoliticaPrestamo> politicas;

    public PoliticaPrestamo para(Usuario usuario) {
        return politicas.stream()
            .filter(p -> p.aplicaA(usuario))
            .findFirst()
    }
}

```

```

        .orElseThrow(() -> new PoliticaNoDefinida(usuario.getTipo()));
    }
}

```

## La prueba que demuestra que el principio se cumplió

Una refactorización OCP no se demuestra con palabras: se demuestra agregando una variante nueva SIN abrir ningún archivo existente. Ese es el pedido real de la Dirección de Bibliotecas.

### ExtensionOcpTest.java PRUEBA DE EXTENSIÓN

```

// Archivo NUEVO. No se modificó ninguna clase anterior.
public class PoliticaEgresado implements PoliticaPrestamo {
    public boolean aplicaA(Usuario u) { return "EGRESADO".equals(u.getTipo()); }
    public int diasPermitidos() { return 5; }
    public int maximoEjemplares() { return 2; }
    public BigDecimal tarifaDiaria() { return new BigDecimal("3.0"); }
}

@Test
@DisplayName("OCP: se agrega EGRESADO sin tocar el código existente")
void extensionSinModificacion() {
    CatalogoPoliticass catalogo = new CatalogoPoliticass(List.of(
        new PoliticaEstudiante(), new PoliticaDocente(),
        new PoliticaAdministrativo(), new PoliticaExterno(),
        new PoliticaEgresado())); // ← única línea agregada

    Usuario juan = new Usuario("205001", "Juan", "juan@usfx.bo", "EGRESADO");

    assertThat(catalogo.para(juan).diasPermitidos()).isEqualTo(5);
    assertThat(catalogo.para(juan).multa(4)).isEqualToByComparingTo("12.0");
}

```

### Terminal DOS COMMITS

```

git commit -m "paso 2 (OCP): Replace Conditional with Polymorphism - PoliticaPrestamo"
git commit -m "paso 2 (OCP): extension - PoliticaEgresado sin modificar código existente"

```

## EVIDENCIA E-2

### Extensión sin modificación

Salida de `git show --stat` del commit de extensión: debe mostrar SOLO archivos nuevos (A), ningún archivo modificado (M) salvo el ensamblado y la prueba.

Captura de `ExtensionOcpTest` en verde.

## Paso 3 · LSP — subtipos que no mienten

| CAMPO           | CONTENIDO                                                                                                                                                                             |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Síntoma         | LibroReferencia lanza UnsupportedOperationException en prestar(); Revista devuelve la misma fecha en renovar(). El cliente CatalogoBiblioteca se defiende con instanceof y try/catch. |
| Principio       | Liskov Substitution Principle — un subtipo debe poder reemplazar a su tipo base sin alterar ninguna propiedad deseable del programa.                                                  |
| Refactorización | Replace Inheritance with Delegation + segregación por rol con sealed interfaces.                                                                                                      |

### El síntoma en el cliente

#### legado/CatalogoBiblioteca.java VIOLA LSP

```
public List<String> prestarTodo(LocalDate hoy) {
    List<String> comprobantes = new ArrayList<>();
    for (MaterialBiblioteca m : materiales) {
        if (m instanceof LibroReferencia) { // ← comprobación de tipo
            continue;
        }
        try {
            comprobantes.add(m.getTitulo() + " -> " + m.prestar(hoy));
        } catch (UnsupportedOperationException e) {
            // "por si acaso": la red de seguridad que delata el mal diseño
        }
    }
    return comprobantes;
}
```

#### LAS TRES REGLAS DE CONTRATO (Tema 3, diapositiva 17)

**Precondiciones:** el subtipo NO puede exigir más que el tipo base.

**Postcondiciones:** el subtipo NO puede prometer menos que el tipo base.

**Invariantes y excepciones:** no se lanzan excepciones nuevas. UnsupportedOperationException en un override es la violación clásica.

### Después

La obra de referencia no es un material prestable: es un material consultable. El error fue heredar para reutilizar dos campos. La solución es modelar los roles, no la taxonomía.

#### dominio/Material.java CUMPLE LSP

```
package bo.edu.usfx.biblioteca.dominio;

// Lo que TODO material tiene
public sealed interface Material
    permits LibroGeneral, Revista, LibroReferencia {
    String signatura();
    String titulo();
}

// Rol: solo lo implementa lo que de verdad sale de la biblioteca
public interface Prestable {
    LocalDate prestar(LocalDate hoy);
}
```

```

public interface Renewable {
    LocalDate renovar(LocalDate limiteActual);
}

public record LibroGeneral(String signatura, String titulo)
    implements Material, Prestable, Renewable {
    public LocalDate prestar(LocalDate hoy) { return hoy.plusDays(7); }
    public LocalDate renovar(LocalDate limiteActual) { return limiteActual.plusDays(7); }
}

public record Revista(String signatura, String titulo)
    implements Material, Prestable { // prestable pero NO renovable
    public LocalDate prestar(LocalDate hoy) { return hoy.plusDays(2); }
}

public record LibroReferencia(String signatura, String titulo)
    implements Material { // ni prestable ni renovable
}

```

#### dominio/Catalogo.java SIN instanceof DEFENSIVO

```

// El cliente ya no comprueba tipos ni atrapa excepciones defensivas
public List<String> prestarTodo(LocalDate hoy) {
    return materiales.stream()
        .filter(Prestable.class::isInstance)
        .map(Prestable.class::cast)
        .map(p -> ((Material) p).titulo() + " -> " + p.prestar(hoy))
        .toList();
}

// Alternativa con pattern matching (Java 21):
public String describir(Material m) {
    return switch (m) {
        case LibroGeneral l -> l.titulo() + " (préstamo 7 días, renovable)";
        case Revista r -> r.titulo() + " (préstamo 2 días)";
        case LibroReferencia x -> x.titulo() + " (solo consulta en sala)";
    }; // exhaustivo: si mañana hay un cuarto material, NO compila
}

```

### SUSTITUYE LAS PRUEBAS, NO LAS BORRES

ContratoLspTest congela hoy el comportamiento roto. Después de este paso esas pruebas ya no se pueden escribir: reescribelas para verificar la sustituibilidad y conserva AMBAS versiones en el informe (la vieja como captura, la nueva como código). Ese contraste es la evidencia E-3.

#### Terminal COMMIT DEL PASO 3

```
git commit -m "paso 3 (LSP): Replace Inheritance with Delegation - roles Prestable y Renewable"
```

### EVIDENCIA E-3

#### Sustituibilidad restablecida

Captura del ContratoLspTest anterior (el que verificaba la excepción) y código de la prueba nueva.

Búsqueda en NetBeans (Ctrl+Shift+F) de "instanceof" y de "UnsupportedOperationException" en el paquete dominio: cero resultados.

## Paso 4 · ISP — nadie depende de lo que no usa

| CAMPO           | CONTENIDO                                                                                                                                               |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Síntoma         | OperacionesBiblioteca declara 7 métodos. EjemplarFisico rechaza 2 (28,6 %) y EjemplarDigital rechaza 3 (42,9 %). Ningún cliente usa la interfaz entera. |
| Principio       | Interface Segregation Principle — ningún cliente debe ser forzado a depender de métodos que no utiliza.                                                 |
| Refactorización | Extract Interface (Fowler) — una interfaz por rol, no por entidad.                                                                                      |

### Antes

#### legado/OperacionesBiblioteca.java VIOLA ISP

```
public interface OperacionesBiblioteca {
    void prestar(String codigoUsuario);
    void devolver(String codigoUsuario);
    void renovar(String codigoUsuario);
    void reservar(String codigoUsuario);
    byte[] descargarPdf(); // solo material digital
    void enviarPorCorreo(String destino); // solo material digital
    void enviarARestauracion(); // solo material físico
}

public class EjemplarFisico implements OperacionesBiblioteca {
    @Override public byte[] descargarPdf() {
        throw new UnsupportedOperationException("Un ejemplar fisico no se descarga");
    }
    ...
}
```

### Después

#### dominio/roles CUMPLE ISP

```
// Una interfaz por rol de cliente
public interface Prestable { void prestar(String codigoUsuario);
                             void devolver(String codigoUsuario); }
public interface Renovable { void renovar(String codigoUsuario); }
public interface Reservable { void reservar(String codigoUsuario); }
public interface Descargable { byte[] descargarPdf(); }
public interface Distribuible { void enviarPorCorreo(String destinatario); }
public interface Restaurable { void enviarARestauracion(); }

public class EjemplarFisico
    implements Prestable, Renovable, Reservable, Restaurable {
    // cuatro métodos, cuatro implementaciones reales, cero excepciones
}

public class EjemplarDigital
    implements Prestable, Descargable, Distribuible {
    // tres métodos, tres implementaciones reales, cero excepciones
}

// Y los clientes piden exactamente lo que usan:
public class MostradorPrestamos {
    public void atender(Prestable material, String codigo) { ... }
}
public class TallerRestauracion {
    public void recibir(Restaurable pieza) { ... }
}
```

### LA MÉTRICA QUE CIERRA EL PASO

Calcula, antes y después, el porcentaje de métodos que cada implementador rechaza. Antes: 28,6 % y 42,9 %. Después debe ser 0 % en ambos. Esa cifra va en tu informe.

**Terminal** COMMIT DEL PASO 4

```
git commit -m "paso 4 (ISP): Extract Interface - roles Prestable, Renovable, Descargable, Restaurable"
```

**EVIDENCIA E-4****Interfaces por rol**

Tabla con el porcentaje de métodos no soportados por implementador, antes y después.

Diagrama de clases del paquete de roles (puedes generarlo con el plugin easyUML de NetBeans).

## Paso 5 · DIP — el dominio define los contratos

| CAMPO           | CONTENIDO                                                                                                                                                                     |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Síntoma         | GestorBiblioteca construye con new su propia ConexionMySQL y su ServidorCorreoSMTP. Probar el cálculo de una multa exigiría una base de datos y un servidor de correo reales. |
| Principio       | Dependency Inversion Principle — los módulos de alto nivel no dependen de los de bajo nivel: ambos dependen de abstracciones.                                                 |
| Refactorización | Replace Constructor with Dependency Injection + puertos y adaptadores.                                                                                                        |

### Antes

#### legado/GestorBiblioteca.java VIOLA DIP

```
public class GestorBiblioteca {

    private final ConexionMySQL conexion =
        new ConexionMySQL("jdbc:mysql://10.0.0.7:3306/biblioteca",
            "root", "usfx2026");    // ← URL y clave en el código

    private final ServidorCorreoSMTP correo =
        new ServidorCorreoSMTP("smtp.usfx.bo", 587);
}
```

### Después

#### dominio/puertos + infraestructura/adaptadores CUMPLE DIP

```
// ----- dominio: los PUERTOS los define el negocio -----
package bo.edu.usfx.biblioteca.dominio;

public interface RepositorioPrestamos {
    void guardar(Prestamo prestamo);
    List<Prestamo> activosDe(Usuario usuario);
}

public interface Notificador {
    void notificar(String destino, String asunto, String mensaje);
}

// ----- infraestructura: los ADAPTADORES -----
package bo.edu.usfx.biblioteca.infraestructura;

public class RepositorioPrestamosJdbc implements RepositorioPrestamos {

    private final DataSource dataSource;

    public RepositorioPrestamosJdbc(DataSource dataSource) {
        this.dataSource = dataSource;    // la URL viene de fuera
    }

    @Override
    public void guardar(Prestamo p) {
        String sql = "INSERT INTO prestamo (codigo_usuario, signatura, fecha, limite)"
            + " VALUES (?, ?, ?, ?)";
        try (Connection c = dataSource.getConnection();
            PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, p.getUsuario().getCodigo());
            ps.setString(2, p.getLibro().getSignatura());
            ps.setObject(3, p.getFechaPrestamo());
            ps.setObject(4, p.getFechaLimite());
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new PersistenciaException(e);
        }
    }
}
```

```

    }
}
}

```

## La recompensa concreta: la prueba corre en milisegundos

Si una clase es difícil de probar, casi siempre es porque construye sus propias dependencias. Invertir la dependencia hace posible el doble de prueba, y el doble de prueba hace posible la prueba unitaria.

### ServicioPrestamosTest.java JUnit 5 + Mockito

```

@ExtendWith(MockitoExtension.class)
class ServicioPrestamosTest {

    @Mock private RepositorioPrestamos repositorio;
    @Mock private Notificador notificador;

    @Test
    @DisplayName("registra el prestamo y notifica al estudiante")
    void registraYNotifica() {
        when(repositorio.activosDe(any())).thenReturn(List.of());
        ServicioPrestamos servicio = new ServicioPrestamos(
            repositorio, notificador, new PoliticaEstudiante());

        Usuario ana = new Usuario("218123", "Ana Quispe", "ana@usfx.bo", "ESTUDIANTE");
        Libro libro = new Libro("005.1 M379c", "Clean Architecture", "R. C. Martin");

        Prestamo prestamo = servicio.registrar(ana, libro, LocalDate.of(2026, 8, 25));

        assertThat(prestamo.getFechaLimite()).isEqualTo(LocalDate.of(2026, 9, 1));
        verify(repositorio).guardar(prestamo);
        verify(notificador).notificar(eq("ana@usfx.bo"), anyString(), anyString());
    }
}

```

## INVERSIÓN ≠ INYECCIÓN

La inversión de dependencias es el principio; la inyección por constructor es solo la técnica más simple para conseguirla. Spring, CDI o Guice la automatizan, pero el principio se cumple igual con un `new` en la clase `Main` — ahí es donde se ensambla todo el sistema.

### Main.java · composition root

```

// Main es el ÚNICO lugar donde el sistema se ensambla
public static void main(String[] args) {
    DataSource ds = configurarDataSource();

    ServicioPrestamos servicio = new ServicioPrestamos(
        new RepositorioPrestamosJdbc(ds),
        new NotificadorSmtip("smtp.usfx.bo", 587),
        new CatalogoPoliticlas(List.of(new PoliticaEstudiante(), ... )));
    ...
}

```

### Terminal COMMIT DEL PASO 5

```
git commit -m "paso 5 (DIP): puertos RepositorioPrestamos y Notificador + inyeccion por constructor"
```



## EVIDENCIA E-5

### **Dependencias invertidas**

Código de la prueba con Mockito y captura del tiempo de ejecución (debe estar por debajo de 100 ms).

Búsqueda de "new ConexionMySQL" y "new ServidorCorreoSMTP" fuera de Main: cero resultados.

## Paso 6 · El diseño como prueba ejecutable

Hasta aquí el diseño depende de la disciplina del equipo. ArchUnit lo convierte en una prueba que falla la compilación cuando alguien lo rompe — incluido tu yo del próximo semestre.

Abre `src/test/java/bo/edu/usfx/biblioteca/arquitectura/ReglasDisenoTest.java` y borra la anotación **@Disabled** de la clase. Ejecuta las pruebas y captura la pantalla EN ROJO: esa captura es la mitad de la evidencia E-6.

```
arquitectura/ReglasDisenoTest.java  ArchUnit

@Test
@DisplayName("R1 - el dominio no conoce la infraestructura (DIP)")
void elDominioNoConoceLaInfraestructura() {
    ArchRule regla = noClasses()
        .that().resideInAPackage("..dominio..")
        .should().dependOnClassesThat()
        .resideInAnyPackage("..infraestructura..", "..persistencia..", "..ui..");

    regla.allowEmptyShould(true).check(CLASES);
}

@Test
@DisplayName("R4 - ninguna clase usa UnsupportedOperationException (LSP / ISP)")
void sinOperacionesNoSoportadas() {
    noClasses().that().resideOutsideOfPackage("..legado..")
        .should().dependOnClassesThat()
        .areAssignableTo(UnsupportedOperationException.class)
        .allowEmptyShould(true).check(CLASES);
}
```

| REGLA | QUÉ VERIFICA                                              | PRINCIPIO                     |
|-------|-----------------------------------------------------------|-------------------------------|
| R1    | El paquete dominio no depende de infraestructura ni de ui | DIP                           |
| R2    | Nadie fuera de infraestructura importa java.sql           | DIP / separación de intereses |
| R3    | Las clases Repositorio* del dominio son interfaces        | DIP                           |
| R4    | Nadie usa UnsupportedOperationException                   | LSP e ISP                     |
| R5    | No existen clases Gestor*, *Manager ni *Utils             | SRP                           |
| R6    | El grafo de paquetes no tiene ciclos                      | ADP                           |

## Métricas de cierre

Completa esta tabla comparando la línea base con el resultado. Los valores de complejidad los obtienes con SonarLint (plugin gratuito de NetBeans) o contando las ramas a mano.

| MÉTRICA                                      | ANTES       | DESPUÉS | META |
|----------------------------------------------|-------------|---------|------|
| Líneas de GestorBiblioteca / clase mayor     | ≈ 190       |         | < 80 |
| Complejidad ciclomática del método mayor     |             |         | < 5  |
| Nº de responsabilidades (actores) por clase  | 4           |         | 1    |
| % de métodos no soportados por implementador | 28,6 / 42,9 |         | 0    |

| MÉTRICA                                | ANTES | DESPUÉS | META  |
|----------------------------------------|-------|---------|-------|
| Reglas de ArchUnit en verde            | 0 / 6 |         | 6 / 6 |
| Pruebas que necesitan BD o SMTP reales | todas |         | 0     |

**EVIDENCIA E-6****El diseño verificado por la máquina**

Par de capturas de ReglasDisenoTest: en rojo antes de terminar, en verde al final.

Tabla de métricas completa.

Salida de git log --oneline --graph con los siete commits.

## 5 PARTE II — EJERCICIO 1: MÓDULO DE SANCIONES Y SOLVENCIAS

| CAMPO                           | DETALLE                                        |
|---------------------------------|------------------------------------------------|
| Paquete                         | bo.edu.usfx.biblioteca.ejercicio1              |
| Clase a refactorizar            | SistemaSanciones.java                          |
| Pruebas que deben seguir verdes | SistemaSancionesTest.java (10 casos)           |
| Principios exigidos             | SRP · OCP · DIP                                |
| Modalidad                       | Parejas                                        |
| Puntaje                         | 40 puntos                                      |
| Entrega                         | Repositorio Git + informe de evidencias en PDF |

### 5.1 Contexto

La Dirección de Bibliotecas sanciona a quien devuelve tarde, daña o pierde un ejemplar. El módulo actual calcula los días de suspensión y el monto de reposición, guarda la sanción en MySQL, avisa por SMS a través de Twilio y emite el certificado de solvencia en HTML. Todo eso vive en una sola clase de cuatro métodos públicos.

#### ejercicio1/SistemaSanciones.java PUNTO DE PARTIDA

```
public String aplicarSancion(String codigoUsuario, String telefono, String tipoInfraccion,
    int diasRetraso, double valorEjemplar, LocalDate hoy) {

    int suspension = calcularSuspension(tipoInfraccion, diasRetraso, valorEjemplar);
    double reposicion = calcularReposicion(tipoInfraccion, valorEjemplar);
    LocalDate habilitado = hoy.plusDays(suspension);

    String sql = "INSERT INTO sancion (codigo, tipo, dias, monto, hasta) VALUES ('"
        + codigoUsuario + "', '" + tipoInfraccion + "', " + suspension + ", "
        + reposicion + ", '" + habilitado + "')";
    System.out.println("[MySQL jdbc:mysql://10.0.0.7:3306/biblioteca] " + sql);

    System.out.println("[SMS Twilio +591" + telefono + "] Suspension de " + suspension + " ...");

    return "ACTA DE SANCION\n"
        + "Usuario      : " + codigoUsuario + "\n" + ...;
}
```

### 5.2 Lo que debes hacer

| # | TAREA       | CRITERIO DE ACEPTACIÓN                                                                                                                                                        |
|---|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Diagnóstico | Tabla con al menos 5 olores de código, su ubicación (archivo : línea) y el principio que viola cada uno.                                                                      |
| 2 | SRP         | Separar cálculo de sanción, persistencia, notificación y emisión de documentos en clases distintas. Ninguna clase resultante debe tener más de un actor.                      |
| 3 | OCP         | Reemplazar las dos cadenas de if/else por una abstracción TipoInfraccion (interfaz, enum con comportamiento o sealed interface). El tope de 180 días debe seguir aplicándose. |

| # | TAREA         | CRITERIO DE ACEPTACIÓN                                                                                                                                   |
|---|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4 | Extensión OCP | Agregar la infracción MORA_REINCIDENTE (120 días de suspensión, reposición igual al 100 % del valor del ejemplar) SIN modificar ninguna clase existente. |
| 5 | DIP           | Definir los puertos Repositorio Sanciones y CanalNotificacion en el dominio y sus adaptadores en infraestructura. Inyectarlos por constructor.           |
| 6 | Pruebas       | Escribir al menos 3 pruebas nuevas con Mockito que verifiquen que se guardó la sanción y que se notificó, sin base de datos ni proveedor de SMS.         |
| 7 | Verificación  | Escribir 2 reglas de ArchUnit propias para este paquete y dejarlas en verde.                                                                             |

### RESTRICCIONES

No puedes borrar, comentar ni relajar ninguna de las 10 pruebas de SistemaSancionesTest. Puedes adaptarlas al nombre nuevo de las clases, pero los valores esperados y el formato del acta deben permanecer idénticos.

Un commit por tarea, con el mensaje en el formato "ejercicio1 tarea N (PRINCIPIO): refactorizacion aplicada".

Prohibido usar Spring, Lombok o cualquier framework: la inyección se hace a mano.

## 5.3 Evidencias exigidas

| CÓD.  | EVIDENCIA                             | FORMATO                                                                          |
|-------|---------------------------------------|----------------------------------------------------------------------------------|
| EJ1-A | Tabla de diagnóstico                  | Tabla en el informe: olor → archivo:línea → principio → refactorización aplicada |
| EJ1-B | Diagrama de clases antes y después    | Dos imágenes generadas con easyUML (NetBeans) o PlantUML                         |
| EJ1-C | Historial de commits                  | Captura de git log --oneline --graph con los 7 commits                           |
| EJ1-D | Suite en verde después de cada commit | Capturas de Test Results (mínimo 3, una por hito)                                |
| EJ1-E | Extensión sin modificación            | git show --stat del commit de MORA_REINCIDENTE: solo archivos nuevos             |
| EJ1-F | Pruebas con dobles                    | Código de las 3 pruebas con Mockito + captura del tiempo de ejecución            |
| EJ1-G | Reglas de ArchUnit                    | Código de las 2 reglas y captura en verde                                        |

## 5.4 Rúbrica del Ejercicio 1 (40 puntos)

| CRITERIO                   | PTOS | DESCRIPTOR DE NIVEL MÁXIMO                                                                                                                                  |
|----------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Diagnóstico                | 5    | Identifica 5 o más olores con ubicación exacta y los asocia correctamente al principio. Distingue síntoma de causa.                                         |
| SRP aplicado               | 7    | Cada clase resultante responde a un solo actor. Los nombres describen la responsabilidad, no la tecnología. Ninguna clase se llama Gestor, Manager o Utils. |
| OCP aplicado               | 8    | No queda ninguna cadena de if/else sobre el tipo de infracción. La abstracción elegida está justificada en el informe.                                      |
| Extensión sin modificación | 5    | MORA_REINCIDENTE se agrega solo con archivos nuevos, demostrado con git show --stat.                                                                        |

| CRITERIO                 | PTOS | DESCRIPTOR DE NIVEL MÁXIMO                                                                                            |
|--------------------------|------|-----------------------------------------------------------------------------------------------------------------------|
| DIP aplicado             | 7    | Los puertos los define el dominio, los adaptadores viven en infraestructura y el ensamblado ocurre en un único punto. |
| Pruebas y comportamiento | 5    | Las 10 pruebas originales siguen verdes, hay 3 pruebas nuevas con dobles y ninguna necesita infraestructura real.     |
| Evidencias y commits     | 3    | Las 7 evidencias están completas y el historial muestra un commit por tarea con mensajes descriptivos.                |

## 6 PARTE III — EJERCICIO 2: RESERVAS DE SALAS Y NOTIFICACIONES

| CAMPO                 | DETALLE                                                                                     |
|-----------------------|---------------------------------------------------------------------------------------------|
| Paquete               | bo.edu.usfx.biblioteca.ejercicio2                                                           |
| Clases a refactorizar | Reserva.java y sus tres subclases · ServicioReservas.java · ServicioReservasBiblioteca.java |
| Pruebas de referencia | ReservasCaracterizacionTest.java (7 casos)                                                  |
| Principios exigidos   | LSP · ISP · DIP                                                                             |
| Modalidad             | Parejas                                                                                     |
| Puntaje               | 40 puntos                                                                                   |
| Entrega               | Repositorio Git + informe de evidencias en PDF                                              |

### 6.1 Contexto

La biblioteca reserva cubículos individuales, salas grupales y el auditorio. La jerarquía actual promete en la clase base algo que dos de sus tres subclases no cumplen, y la interfaz del servicio mezcla el ciclo de vida de la reserva con tres canales de notificación y dos formatos de comprobante.

#### ejercicio2 · las tres violaciones de contrato PUNTO DE PARTIDA

```
/** Contrato de la clase base: 1 <= horas <= 8. Devuelve la hora de fin. */
public abstract LocalDateTime confirmar(int horas);

/** Contrato: cancelar siempre es posible antes del inicio. */
public abstract void cancelar();

// ---- pero ----
public class ReservaSalaGrupal extends Reserva {
    @Override public LocalDateTime confirmar(int horas) {
        if (horas > 4) throw new IllegalArgumentException(...); // precondición ENDURECIDA
        if (cantidadPersonas < 3) throw new IllegalStateException(...);
        return inicio.plusHours(horas);
    }
    @Override public void cancelar() {
        throw new UnsupportedOperationException(...); // postcondición DEBILITADA
    }
}

public class ReservaAuditorio extends Reserva {
    @Override public LocalDateTime confirmar(int horas) {
        throw new UnsupportedOperationException("requiere autorizacion");
    }
    @Override public void cancelar() { /* silencio: no hace nada */ }
}
```

### 6.2 Lo que debes hacer

| # | TAREA                   | CRITERIO DE ACEPTACIÓN                                                                                                                       |
|---|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Diagnóstico de contrato | Para cada subclase, indica cuál de las tres reglas de Liskov rompe: precondición, postcondición o invariante/exceptión. Con la línea exacta. |

| # | TAREA                     | CRITERIO DE ACEPTACIÓN                                                                                                                                                          |
|---|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 | LSP                       | Rediseñar la jerarquía para que cualquier subtipo sea sustituible. El auditorio NO es una reserva confirmable por el usuario: modela ese hecho en el tipo, no en una excepción. |
| 3 | Prueba de sustituibilidad | Escribir una prueba parametrizada que recorra TODOS los subtipos confirmables y verifique el mismo contrato en todos, sin instanceof ni try/catch.                              |
| 4 | ISP                       | Partir ServicioReservas (8 métodos) en interfaces por rol. Ningún implementador debe declarar un método que no pueda cumplir.                                                   |
| 5 | Canales de notificación   | Modelar los tres canales (correo, SMS, WhatsApp) detrás de una sola abstracción, de modo que agregar un cuarto canal no obligue a editar el servicio.                           |
| 6 | DIP                       | Injectar el repositorio y los canales por constructor. El servicio de reservas no debe conocer ninguna tecnología concreta.                                                     |
| 7 | Verificación              | Regla de ArchUnit que prohíba UnsupportedOperationException en todo el paquete, en verde.                                                                                       |

### LA PREGUNTA DE DISEÑO QUE DEBES RESPONDER EN EL INFORME

¿El auditorio debe seguir siendo un subtipo de Reserva? Justifica tu decisión con el criterio de sustituibilidad, no con la taxonomía del mundo real. Recuerda el caso Rectángulo/Cuadrado del Tema 3: un cuadrado ES un rectángulo en geometría, pero no como tipo mutable.

## 6.3 Evidencias exigidas

| CÓD.  | EVIDENCIA                        | FORMATO                                                                             |
|-------|----------------------------------|-------------------------------------------------------------------------------------|
| EJ2-A | Tabla de violaciones de contrato | Subclase → regla de Liskov rota → línea → corrección aplicada                       |
| EJ2-B | Jerarquía antes y después        | Dos diagramas de clases (easyUML o PlantUML)                                        |
| EJ2-C | Prueba de sustituibilidad        | Código de la prueba parametrizada + captura en verde                                |
| EJ2-D | Métrica ISP                      | Tabla: métodos declarados vs. métodos rechazados por implementador, antes y después |
| EJ2-E | Cuarto canal de notificación     | Commit que agrega un canal nuevo solo con archivos nuevos                           |
| EJ2-F | Historial de commits             | git log --oneline --graph con los 7 commits                                         |
| EJ2-G | ArchUnit                         | Captura en rojo y en verde de la regla sin UnsupportedOperationException            |
| EJ2-H | Justificación de diseño          | Media página respondiendo la pregunta del recuadro anterior                         |

## 6.4 Rúbrica del Ejercicio 2 (40 puntos)

| CRITERIO                | PTOS | DESCRIPTOR DE NIVEL MÁXIMO                                                                                                |
|-------------------------|------|---------------------------------------------------------------------------------------------------------------------------|
| Diagnóstico de contrato | 6    | Identifica correctamente las tres violaciones y nombra la regla de Liskov rota en cada caso.                              |
| LSP aplicado            | 9    | Ningún subtipo lanza excepciones nuevas ni endurece precondiciones. El auditorio se modela sin mentir sobre su capacidad. |



| CRITERIO                     | PTOS | DESCRIPTOR DE NIVEL MÁXIMO                                                                  |
|------------------------------|------|---------------------------------------------------------------------------------------------|
| Prueba de sustituibilidad    | 5    | Prueba parametrizada que recorre todos los subtipos con el mismo contrato. Cero instanceof. |
| ISP aplicado                 | 8    | Interfaces por rol, coherentes y con nombres del dominio. Cero métodos no soportados.       |
| Abstracción de canales + DIP | 7    | Un cuarto canal se agrega sin editar el servicio. Los puertos los define el dominio.        |
| Evidencias y commits         | 3    | Las 8 evidencias completas, un commit por tarea.                                            |
| Justificación de diseño      | 2    | Argumenta desde la sustituibilidad y reconoce el intercambio elegido.                       |

## 7 EL INFORME DE EVIDENCIAS

La refactorización no se evalúa leyendo el código final: se evalúa leyendo el camino. El informe es el documento donde ese camino queda demostrado.

### 7.1 Estructura obligatoria

| SEC. | CONTENIDO           | EXTENSIÓN Y FORMATO                                                                                                                      |
|------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Carátula            | Materia, tema, integrantes con código, fecha, URL del repositorio                                                                        |
| 2    | Diagnóstico inicial | Tabla olor → archivo:línea → principio → refactorización prevista                                                                        |
| 3    | Bitácora de pasos   | Un bloque por paso: qué cambió, por qué, captura de las pruebas en verde                                                                 |
| 4    | Diagramas           | Clases antes y después; opcionalmente secuencia del caso de uso principal                                                                |
| 5    | Evidencias          | Todas las capturas numeradas E-0 a E-6 / EJ1-A a EJ1-G / EJ2-A a EJ2-H                                                                   |
| 6    | Métricas            | Tabla comparativa antes / después / meta                                                                                                 |
| 7    | Reflexión           | Media página: qué principio costó más aplicar y por qué; qué NO refactorizaron y por qué (YAGNI cuenta como respuesta válida y correcta) |
| 8    | Anexo               | Salida completa de git log --oneline --graph                                                                                             |

### 7.2 Formato de los mensajes de commit

#### Plantilla de mensajes

paso 1 (SRP): Extract Class - separar dominio, persistencia, notificación y presentación  
 paso 2 (OCP): Replace Conditional with Polymorphism - PolíticaPrestamo  
 paso 3 (LSP): Replace Inheritance with Delegation - roles Prestable y Renovable  
 paso 4 (ISP): Extract Interface - roles por cliente  
 paso 5 (DIP): puertos + inyección por constructor  
 paso 6: reglas ArchUnit en verde

ejercicio1 tarea 3 (OCP): TipoInfraccion como sealed interface  
 ejercicio2 tarea 2 (LSP): jerarquía de reservas por rol

#### CÓMO SE DETECTA UNA ENTREGA SIN TRABAJO REAL

Un único commit con 40 archivos cambiados no es una refactorización: es una reescritura. Un historial donde las pruebas nunca estuvieron en rojo tampoco es creíble en el Paso 6. La bitácora y el historial deben contar la misma historia.

### 7.3 Uso de asistentes de IA

Está permitido usar asistentes de IA como apoyo, con dos condiciones: (1) declararlo en la sección de reflexión indicando en qué pasos se usó y para qué; (2) poder explicar oralmente cualquier línea del código entregado. En la defensa se pregunta por decisiones, no por sintaxis: por qué elegiste una sealed interface en lugar de un enum, o por qué esa abstracción y no otra.

## 8 EVALUACIÓN GLOBAL Y CALENDARIO

| COMPONENTE                                           | PUNTOS | ENTREGA                          |
|------------------------------------------------------|--------|----------------------------------|
| Práctica guiada (pasos 0 a 6 + evidencias E-0 a E-6) | 20     | Al final de la sesión guiada     |
| Ejercicio 1 — módulo de sanciones                    | 40     | Sesión de laboratorio 1 + 7 días |
| Ejercicio 2 — reservas y notificaciones              | 40     | Sesión de laboratorio 2 + 7 días |
| TOTAL                                                | 100    |                                  |

### 8.1 Niveles de desempeño

| NIVEL         | RANGO    | DESCRIPCIÓN                                                                                                                                               |
|---------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sobresaliente | 91 – 100 | Aplica los cinco principios con criterio, justifica dónde NO aplicarlos, y las reglas de ArchUnit y las pruebas demuestran objetivamente cada afirmación. |
| Competente    | 71 – 90  | Aplica los principios correctamente. La evidencia está completa pero la justificación de las decisiones es superficial.                                   |
| En desarrollo | 51 – 70  | Refactoriza parcialmente. Quedan cadenas de if/else o excepciones no soportadas. La evidencia es incompleta.                                              |
| Insuficiente  | 0 – 50   | El comportamiento observable cambió, las pruebas de caracterización están rojas o modificadas, o no hay historial de commits.                             |

#### EL ERROR MÁS CARO DE ESTA PRÁCTICA

Aplicar SOLID "al 100 %" produce complejidad innecesaria — que es, ella misma, uno de los siete síntomas de mal diseño. Una interfaz con un solo implementador y ninguna prueba que la sustituya no es DIP: es indirección decorativa. Si en tu informe justificas por qué decidiste NO abstraer algo, eso suma puntos, no los resta.

## 9 ANEXOS

### A. Tabla de referencia rápida

|   | PRINCIPIO                 | SEÑAL DE VIOLACIÓN                                                                                                   | REFACTORIZACIÓN                                          |
|---|---------------------------|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| S | Responsabilidad única     | La clase se llama Gestor, Manager o Utils; su descripción necesita la palabra "y"; dos equipos la editan cada sprint | Extract Class · Move Method                              |
| O | Abierto / cerrado         | switch o if/else sobre un "tipo" que crece cada vez que el negocio agrega un caso                                    | Replace Conditional with Polymorphism · Strategy         |
| L | Sustitución de Liskov     | UnsupportedOperationException en un override; instanceof en el código cliente                                        | Replace Inheritance with Delegation · interfaces por rol |
| I | Segregación de interfaces | Implementadores que rechazan la mitad de los métodos                                                                 | Extract Interface por rol de cliente                     |
| D | Inversión de dependencias | new de una clase de infraestructura dentro de la lógica de negocio; pruebas que exigen BD real                       | Dependency Injection · puertos y adaptadores             |

### B. Comandos de referencia

```
# --- Maven ---
mvn clean test                ejecuta toda la suite
mvn -Dtest=SistemaSancionesTest test  ejecuta una sola clase de prueba
mvn surefire-report:report          genera target/site/surefire-report.html
mvn compile exec:java -Dexec.mainClass=bo.edu.usfx.biblioteca.Main

# --- Git ---
git log --oneline --graph      historial compacto (evidencia)
git show --stat <hash>        archivos tocados por un commit
git diff --stat paso0..HEAD    resumen de todo el trabajo
git tag paso1 && git tag paso2  marcar hitos
```

### C. Atajos útiles de NetBeans

| ACCIÓN                         | ATAJO        | PARA QUÉ SIRVE EN ESTA PRÁCTICA                            |
|--------------------------------|--------------|------------------------------------------------------------|
| Refactor → Extract Interface   | Alt+Shift+I  | Pasos 4 y 5: crear el puerto a partir de la clase concreta |
| Refactor → Extract Method      | Alt+Shift+M  | Paso 1: aislar cada bloque antes de moverlo                |
| Refactor → Move Class          | Alt+Shift+V  | Paso 1: llevar la clase al paquete correcto                |
| Refactor → Introduce Parameter | Alt+Shift+P  | Paso 5: convertir un new interno en un parámetro           |
| Find Usages                    | Alt+F7       | Ver el impacto real antes de mover algo                    |
| Buscar en el proyecto          | Ctrl+Shift+F | Evidencias: buscar "instanceof", "new ConexionMySQL"       |
| Ejecutar pruebas               | Alt+F6       | Después de cada paso, sin excepción                        |
| Insertar código (Generate)     | Alt+Insert   | Constructores para la inyección de dependencias            |

## D. Bibliografía

- IEEE Computer Society. SWEBOK Guide v4.0, 2024 — Área de Conocimiento 2: Software Design.
- Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.
- Martin, R. C. Agile Software Development: Principles, Patterns, and Practices. Prentice Hall, 2002.
- Martin, R. C. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008.
- Fowler, M. Refactoring: Improving the Design of Existing Code, 2.ª ed. Addison-Wesley, 2018.
- Feathers, M. Working Effectively with Legacy Code. Prentice Hall, 2004 — pruebas de caracterización.
- Liskov, B. y Wing, J. «A Behavioral Notion of Subtyping». ACM TOPLAS 16(6), 1994.
- Meyer, B. Object-Oriented Software Construction. Prentice Hall, 1988 — OCP y diseño por contrato.
- Larman, C. Applying UML and Patterns, 3.ª ed. Prentice Hall, 2004 — GRASP.
- Documentación oficial: ArchUnit ([archunit.org](http://archunit.org)), JUnit 5 ([junit.org](http://junit.org)), Mockito ([site.mockito.org](http://site.mockito.org)).

### SÍNTESIS

Diseñar es decidir dónde poner cada responsabilidad y qué depende de qué. Bajo acoplamiento y alta cohesión son el criterio maestro; SOLID es la forma concreta de perseguirlo. Refactorizar exige pruebas: sin red de seguridad no hay refactorización posible, solo reescritura con los ojos vendados.